object. Additionally, as this handler corresponds to an HTTP `POST` request, the `Input` property is also model-bound to the request.

NOTE Unlike most .NET classes, you can't use method overloading to have multiple page handlers on a Razor Page with the same name.

When a page handler uses model-bound properties or parameters, it should always check that the provided model is valid using `ModelState.IsValid`. The `ModelState` property is exposed as a property on the base `PageModel` class and can be used to check that all the bound properties and parameters are valid. You'll see how the process works in chapter 16 when you learn about validation.

Once a page handler establishes that the arguments provided to a page handler method are valid, it can execute the appropriate business logic and handle the request. In the case of the `OnPost` handler, this involves calling the injected `SearchService` and setting the result on the `Results` property. Finally, the handler returns a `PageResult` by calling the helper method on the `PageModel` base class:

```
return Page();
```

If the model isn't valid, as indicated by `ModelState.IsValid`, you don't have any results to display! In this example, the action returns a `RedirectToPageResult` using the `RedirectToPage()` helper method. When executed, this result sends a `302` Redirect response to the user, which will cause their browser to navigate to the `Index` Razor Page.